🕮 Techniques for Handling Missing Data

When working with real-world datasets, you will often encounter missing values that can hinder your analysis or model performance. Handling missing data is a crucial data cleaning step. There are several strategies you can use to address missing values in your data. The most straightforward approach is to remove rows or columns containing missing values using the **dropping method**. While this ensures only complete data is used, it can reduce your dataset size and potentially remove valuable information.

Another common technique is to **fill missing values with a constant**, such as zero or an empty string, which can be useful for categorical or indicator columns. However, this may introduce bias if the constant does not represent the true nature of the missing data.

A more nuanced approach is **statistical imputation**, where missing values are replaced with statistics calculated from the available data. For numerical columns, you might use the mean or median value of the column. The mean is best suited for columns with a normal (symmetric) distribution, while the median is more robust for skewed distributions or when outliers are present.

```python
import pandas as pd

# Create a sample DataFrame with missing values
data = {
    "age": [25, None, 30, 22, None],
    "income": [50000, 60000, None, 52000, 58000]
}
df = pd.DataFrame(data)

# Drop rows with any missing values
df_dropped = df.dropna()

# Fill missing values with a constant (e.g., 0)
df_filled_constant = df.fillna(0)

# Impute missing values in 'age' column with the mean
df_filled_mean = df.copy()
df_filled_mean["age"] =
df_filled_mean["age"].fillna(df_filled_mean["age"].mean())

print("Original DataFrame:")
print(df)
print("\nAfter dropna():")
print(df_dropped)
print("\nAfter fillna(0):")
print(df_filled_constant)
print("\nAfter fillna() with mean for 'age':")
print(df_filled_mean)
```